FPT UNIVERSITY | EMBEDDED LINUX SYSTEMS

ASSIGNMENT 01

HOW TO WRITE A DEVICE DRIVER

IOCTL Character Device | Misc Counter Device
System Architecture, Design Analysis and Verification

STUDENT	COURSE / ENVIRONMENT
Nguyen Xuan Phu **Student ID:** SE203361	LOS201 - Embedded Linux Systems Linux 5.15.0 / ARMv7 / QEMU vexpress-a9

Report prepared: 25 October 2026

Technical report draft for review against the student's implementation and logs

Executive Summary and Report Map

This report examines two Linux 5.15 kernel modules implemented and tested under QEMU on an ARMv7 vexpress-a9 target. Task 1 extends a character driver with four **ioctl** commands; Task 2 creates a separate miscellaneous counter device. The analysis relates the driver design to userspace, the virtual file system (VFS), synchronization, and the procfs/sysfs interfaces.

Verification snapshot

DELIVERABLE	OBSERVED EVIDENCE
Task 1 - asgn1_driver	ARM module compiled; **ALL IOCTL TESTS PASSED**; major number 241.
Task 2 - counter_driver	ARM module compiled; **ALL COUNTER DRIVER TESTS PASSED**; misc major 10, observed minor 125.
Kernel diagnostics	Both modules log successful load and unload; logs captured from the QEMU session.
Testing environment	Linux 5.15.0, armv7l; static ARM userspace test for Task 1.

Report contents

1. Architecture and data paths
2. Task 1 - IOCTL character driver
3. Task 2 - miscellaneous counter driver
4. Character / block / network driver comparison
5. Engineering decisions and concurrency
6. ioctl encoding and safe userspace exchange
7. Task 1 and Task 2 test evidence
8. Debugging notes and validation limits
9. Task 4 theory questions
10. Submission evidence and references

> **Evidence policy**
> Observed outputs are reproduced only where they appear in the student's terminal results. Assertions not covered by the supplied tests are identified as analysis or recommendations, not as completed tests.

1. Integrated Driver Architecture

The following diagram summarizes both drivers and their principal interfaces. The students' test programs enter the kernel via system calls; VFS resolves the opened device and dispatches to the registered **file_operations**. Device registration exposes the character-device nodes; procfs and sysfs provide observational interfaces.

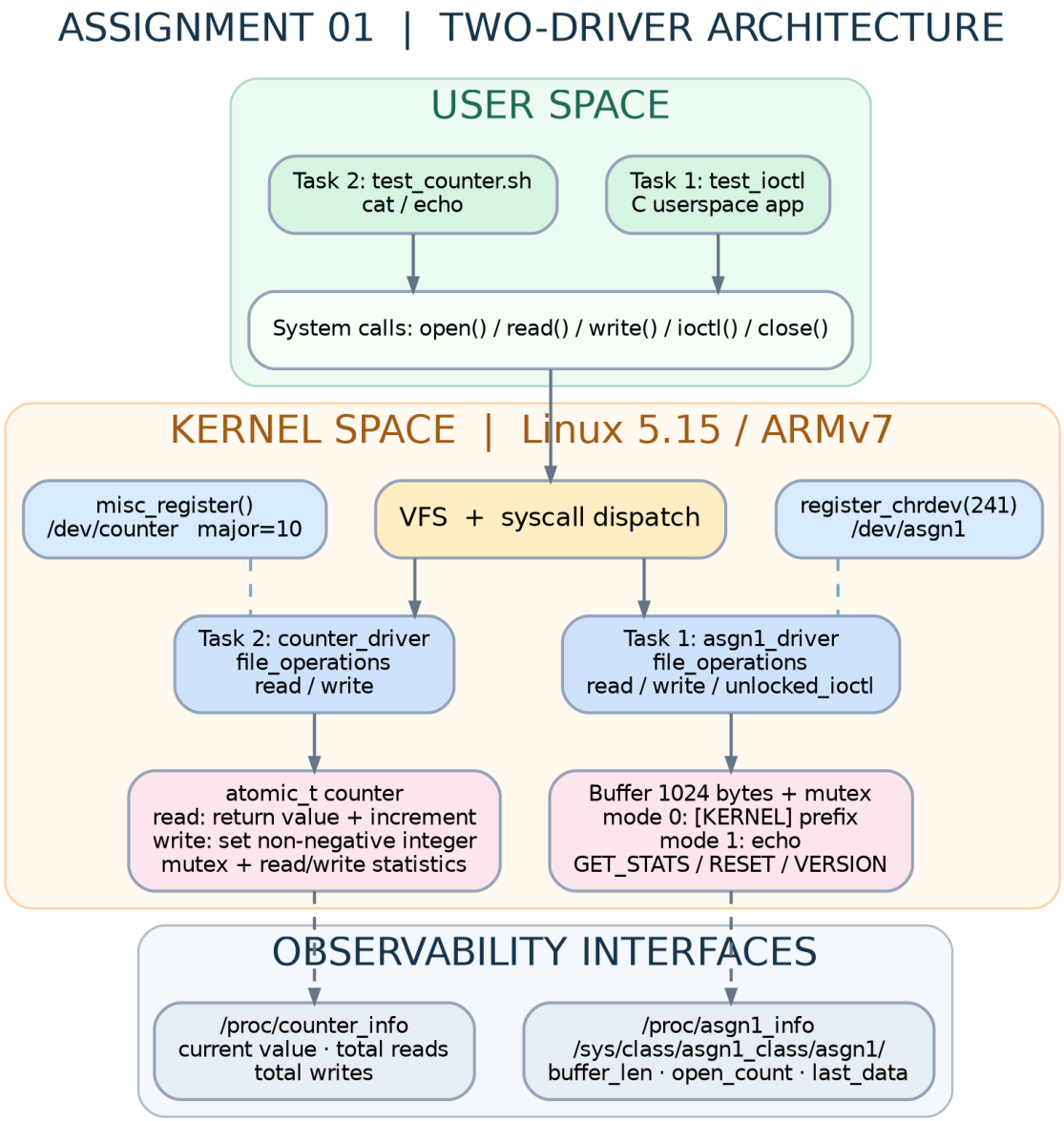


Figure 1. Two-driver Linux architecture. Redrawn schematic from the confirmed implementation details.

Required ioctl path: test_ioctl ioctl() system call VFS file_operations.unlocked_ioctl command handler. For **SET_MODE**, the handler calls **copy_from_user()**; for **GET_STATS** and **GET_VERSION**, it calls **copy_to_user()**. The diagram must be reviewed/redrawn in the student's own style when required by the course.

2. Task 1 - IOCTL Character Device

2.1. Registration and interfaces

The **asgn1_driver** module registers a character device with **register_chrdev(241, "asgn1", ...)**, defines its **file_operations**, and publishes the device as **/dev/asgn1**. The driver maintains a fixed-size 1,024-byte kernel buffer; metadata include open, read and write counters plus the last write size. A mutex guards shared state during read, write, and ioctl updates.

2.2. Four ioctl commands

COMMAND	BEHAVIOUR
ASGN1_RESET_BUFFER	Clears the buffer and resets **read_count**, **write_count** and **last_write_size** to zero.
ASGN1_GET_STATS	Returns **open_count**, **write_count**, **read_count**, **buffer_len** and **last_write_size**.
ASGN1_SET_MODE	Mode 0 prepends **[KERNEL]**; Mode 1 stores original data. Invalid values are rejected.
ASGN1_GET_VERSION	Returns a version string containing the module version and student ID; userspace buffer is 64 bytes.

2.3. Data and observability

On successful write, the driver stores either the prefixed or unchanged payload and updates the length and counters. Reads copy data to userspace and advance the file offset; subsequent reads return EOF once data has been exhausted. Users can inspect **/proc/asgn1_info** and read-only sysfs entries named **buffer_len**, **open_count**, and **last_data**.

> **Important implementation detail**
> The default operating mode is Mode 1 (Echo). A successful RESET_BUFFER does not reset open_count, because the assignment specifically requests resetting read_count and write_count.

3. Task 2 - Misc Counter Device

3.1. Device registration

The independent **counter_driver** module uses the Linux miscellaneous-device framework. A **struct miscdevice** registered with **misc_register()** names the device **counter**. It uses **major number 10** and a dynamically allocated minor number: the observed QEMU run assigned **minor 125**. No fixed minor value is assumed in the design.

3.2. Read and write semantics

Reading **/dev/counter** returns the current counter as a decimal ASCII string followed by a newline. The first successful read of an open file increments the counter and read statistic; per-open state and the file offset preserve correct handling of short reads and eventual EOF. Writing a valid non-negative ASCII integer uses **kstrtoint()** and updates the counter; invalid or negative strings return **-EINVAL**.

```
cat /dev/counter  -> 0
cat /dev/counter  -> 1
cat /dev/counter  -> 2
echo 100 > /dev/counter
cat /dev/counter  -> 100
```

3.3. Statistics and cleanup

The counter and operation statistics use **atomic_t** values; a mutex protects compound operations. **/proc/counter_info** exposes the current value, total reads and total writes. Module initialization creates both the misc device and procfs entry; the unload path removes the procfs entry and deregisters the misc device.

> **Device-node caveat**
> On the tested initramfs, registering the device did not automatically create /dev/counter. The test script obtained the registered minor from /proc/misc and created a character node with mknod. This is an environment/device-node setup issue, not evidence of a failed misc_register().

4. Comparison of Linux Driver Types

The appropriate driver subsystem is selected according to the data abstraction and the kernel interface required by the hardware or virtual device.

CRITERION	CHARACTER	BLOCK	NETWORK
Kernel API	file_operations	block_device_operations + block I/O	net_device_ops + network stack
Data unit	Byte stream / records	Sectors or logical blocks	Packets / frames
Typical access	Sequential; device-specific	Random-access persistent storage	Packet transmission / reception
Buffering	Driver buffers / optional kernel buffering	Page cache; block request queues	sk_buff and TX/RX queues
Examples	UART, GPIO, virtual counter	eMMC, SD card, SSD	Ethernet MAC, Wi-Fi adapter
Selection	Simple control and data exchange	Filesystem-backed storage	Network connectivity

4.1. Design choice examples

Example A - Virtual counter: The counter exports a small integer value and accepts a control write. It does not expose addressable sectors or network frames. A miscellaneous character driver is the simplest suitable design.

Example B - eMMC storage: An embedded board storing an operating system and application files benefits from the block layer and its request handling and filesystem integration. A generic character driver would reimplement significant storage functionality.

Example C - Ethernet interface: When a board must transmit and receive Ethernet frames, a network driver integrates with the Linux network stack and the **net_device_ops** interface.

5. Driver Registration and Engineering Choices

5.1. register_chrdev() versus misc_register()

DESIGN QUESTION	TASK 1 - CHARACTER DEVICE	TASK 2 - MISC DEVICE
Registration	register_chrdev() with explicit major 241	misc_register() under shared major 10
Device node	/dev/asgn1 via device_create(); node may require devtmpfs or mknod	/dev/counter with dynamic minor; node may require devtmpfs or mknod
Advantages	Explicit character-device registration and control over major number	Less registration boilerplate; natural for small devices
Trade-offs	More manual class/device lifecycle and cleanup	Misc framework is specific to small character-style devices
Selection rationale	Assignment exercises direct character registration + ioctl	A single-purpose integer counter suits miscdevice

5.2. Why not reuse Task 1?

Task 2 was specified as an independent driver. Although both modules are exposed through character-device semantics, the counter uses a different registration mechanism and data model. Separate modules allow their interfaces, state, and cleanup to be tested independently.

5.3. Resources and lifecycle

Task 1 registers a character device, class/device objects, sysfs attributes and a procfs entry; its failure and unload paths must release each resource. Task 2 registers a miscdevice and procfs entry and reverses both operations during unload. The observed unload logs verify that each module's exit path was reached; full resource-leak testing would require additional checks.

6. Concurrency, Race Conditions and ioctl Encoding

6.1. atomic_t and compound operations

A plain **counter++** is a read-modify-write sequence. Two threads can read the same old value, both compute old + 1, and store the same new value, losing one increment. Linux **atomic_inc()** avoids this lost-update problem for one counter update. However, **atomic_read()** followed later by **atomic_inc()** is not automatically one indivisible transaction; the Task 2 mutex serializes this compound operation.

6.2. Mutex and userspace copying

In Task 1, a mutex guards the kernel buffer and its statistics. **copy_from_user()** validates and copies bytes from userspace; **copy_to_user()** copies a result back to userspace. These functions are appropriate to kernel/userspace boundaries because userspace addresses can be inaccessible or fault. Invalid transfers return errors instead of assuming a raw pointer can be dereferenced.

6.3. ioctl magic number and layout

The shared header assigns magic character **'A'** and command numbers 0 to 3. The **_IO** macro represents no argument, **_IOR** encodes a kernel-to-user result and **_IOW** encodes a user-to-kernel argument. The direction is described from the userspace caller's perspective.

```
#define ASGN1_IOC_MAGIC 'A'
#define ASGN1_RESET_BUFFER _IO('A', 0)
#define ASGN1_GET_STATS _IOR('A', 1, struct asgn1_stats)
#define ASGN1_SET_MODE  _IOW('A', 2, int)
#define ASGN1_GET_VERSION _IOR('A', 3, char[64])
```

_IOC_NR(ASGN1_GET_STATS) is **1**; **_IOC_SIZE(ASGN1_GET_STATS)** is **sizeof(struct asgn1_stats)** (20 bytes for the five 32-bit int fields in the tested ABI). For the version command, **_IOC_SIZE** is **64**. Magic numbers help organize command namespaces but do not replace access checks.

7. Task 1 - Test Procedure and Evidence

7.1. Target and execution

The compiled ARM module **asgn1_driver.ko** was inserted into QEMU Linux 5.15.0, and the statically linked ARM executable **test_ioctl** exercised the four ioctl commands and edge cases. Since /dev/asgn1 did not appear automatically, the device node was created manually using the registered major 241.

7.2. Selected observed results

TEST	TERMINAL EVIDENCE	STATUS
GET_VERSION	asgn1_driver v1.0 - SE203361	PASS
Mode 0 - Kernel	Expected [[KERNEL]Hello]; Actual [[KERNEL]Hello]	PASS
Mode 1 - Echo	Expected [World]; Actual [World]	PASS
GET_STATS	Open 3; write 2; read 2; buffer_len 5; last_write_size 5	PASS
RESET_BUFFER	Write 0; read 0; buffer length 0; empty read	PASS
Invalid mode / ioctl	Mode 2 rejected; unsupported command rejected	PASS
Module lifecycle	insmod and rmmod succeeded; procfs available during test	PASS

7.3. Log evidence

```
asgn1: SET_MODE = 0 (Kernel mode)
asgn1: write success, mode=0, length=13
asgn1: SET_MODE = 1 (Echo mode)
asgn1: write success, mode=1, length=5
asgn1: GET_STATS called
asgn1: RESET_BUFFER called
asgn1: module unloaded successfully
```

The automated **test_asgn1.sh** output included **ALL ASGN1 DRIVER TESTS PASSED**. The corresponding transcript is retained as **logs/test_asgn1_output.txt**.

8. Task 2 - Test Procedure and Evidence

8.1. Test sequence

The **test_counter.sh** script inserted **counter_driver.ko**, confirmed the misc registration, created /dev/counter if needed, read the counter three times, wrote 100, reset to 0, rejected malformed input, validated procfs totals, and removed the module.

MILESTONE	OBSERVED RESULT
Initial /proc/counter_info	Current counter 0; total reads 0; total writes 0
Read sequence	0 1 2; each read passed
After echo 100	Current counter 100; total reads 3; total writes 1
Read after echo 100	Returned 100; read count increased
After echo 0	Current counter 0; total reads 4; total writes 2
Final after read 0	Current counter 1; total reads 5; total writes 2
Invalid input	-5 and abc both produced Invalid argument
Unloading	procfs removed; kernel reported module unloaded

8.2. Selected transcript

```
counter_driver: /dev/counter registered
counter_driver: /proc/counter_info created
Expected: 0 | Actual: 0
Expected: 1 | Actual: 1
Expected: 2 | Actual: 2
Expected: 100 | Actual: 100
[PASS] Final statistics
ALL COUNTER DRIVER TESTS PASSED
```

Detailed userland evidence is kept in **logs/test_counter_output.txt**; kernel log data for both drivers are stored in **logs/dmesg_full.txt**.

9. Debugging Notes and Verification Limits

9.1. Observed problem: missing device node

After loading the modules successfully, QEMU reported that **/dev/counter** and, separately, **/dev/asgn1** did not exist. Registration was not the problem: **/proc/misc** contained **125 counter**, **/proc/devices** contained **241 asgn1**, and both modules were visible in **lsmod**.

The initramfs environment did not create these device nodes automatically. The working solution was to create nodes matching the kernel-assigned major/minor. For misc devices, the minor was read dynamically rather than hard-coded.

```
# Inside QEMU as root
minor=$(awk '$2=="counter" {print $1}' /proc/misc)
mknod /dev/counter c 10 "$minor"
mknod /dev/asgn1 c 241 0
```

9.2. Verified correction

Following node creation, read/write/ioctl tests passed. Manual device nodes were deleted after module removal. In a production root filesystem, devtmpfs/udev integration should normally automate node management.

9.3. Test coverage and limitations

The supplied logs provide strong functional evidence for the specified paths, including successful load/unload, command responses and invalid-input handling. They do **not** demonstrate formal multi-process stress tests, adversarial buffer-length tests, kernel memory-leak analysis, or persistence after reboot. These would be appropriate extensions but are not claimed as completed in this report.

> **Practical reporting rule**
> Use the full raw test logs as attachments. Selected excerpts inside this report are summaries; they do not replace the underlying terminal evidence.

10. Task 4 - Theory Questions (Part I)

Question 1. Module reference counting

Linux kernel modules use reference counting to prevent a module from being removed while code in that module is still in use. A successful open of a device whose file_operations has **.owner = THIS_MODULE** normally holds a module reference, and close/release drops it. Other kernel users can explicitly obtain and release references through **try_module_get()/module_put()**.

If a process still has the device open, ordinary **rmmod** typically reports that the module is in use and refuses removal. The safe response is to identify and close the open file descriptor or terminate the using process in a controlled way, then retry unload. Some kernels support forced unloading with **CONFIG_MODULE_FORCE_UNLOAD** and an appropriate userspace option, but it is dangerous: active code/data may disappear and the kernel may crash. It should not be the normal lab cleanup method.

Question 2. Why copy_to_user() instead of memcpy()?

The pointer supplied to **ASGN1_GET_STATS** refers to a **userspace virtual address**, not a trusted kernel buffer. Kernel code cannot assume that the pointer is mapped, accessible, or stable. Directly dereferencing it or using raw **memcpy()** can fault or bypass required user-access handling. The memory management unit (MMU) applies mappings and permission checks; a user pointer may be unmapped or mapped without the required access, and a page fault may occur.

copy_to_user() is the kernel API that performs the controlled copy and handles access faults by reporting the number of uncopied bytes. The driver converts a failed copy into **-EFAULT**. Using the correct API helps keep kernel and userspace address-space handling safe and predictable.

11. Task 4 - Theory Questions (Part II)

Question 3. MTD versus block subsystem

The **Memory Technology Device (MTD)** subsystem is used primarily for raw flash devices such as raw NAND or NOR, where erase-block boundaries, bad-block management, error-correcting codes and flash-specific constraints matter. Conventional block devices, including eMMC and SSDs, present addressable logical sectors; a storage controller often hides raw flash erase and wear details.

For example, a board with **raw NAND flash** used for boot firmware and a JFFS2/UBIFS-compatible storage design requires MTD-aware handling. By contrast, an **eMMC module** presents block storage and is normally managed through the block layer. Treating raw NAND as an ordinary disk without considering bad blocks and erase constraints would be a poor design choice.

Question 4. Why read() must handle *ppos

A command such as **cat /dev/counter** repeatedly invokes **read()** through the VFS until it receives **0 bytes (EOF)**, an unrecoverable error, or is otherwise stopped. If the driver always returns a nonempty counter string without advancing or inspecting the position, cat can print indefinitely because every read appears to have more data.

The driver therefore checks ***ppos**, returns the initial value once per open file, advances the position by the bytes successfully copied, and returns 0 after all bytes have been read. Per-open state also avoids counting an incremental second fragment of a short read as a brand-new counter observation.

Key distinction

> **Single successful read versus repeated VFS requests**
> "Auto-increment on read" is a logical device operation, not an instruction to increment on every partial copy. A correct EOF strategy prevents an unbounded cat loop and ensures predictable statistics.

12. Evidence, Submission Layout and References

12.1. Files that substantiate this report

PATH IN ASSIGNMENT	PURPOSE
Task1_IOCTL/asgn1_driver.c / asgn1_ioctl.h	Driver and ioctl definitions
Task1_IOCTL/test_ioctl.c / test_asgn1.sh	Task 1 userspace and shell tests
Task2_Misc/counter_driver.c / test_counter.sh	Counter misc driver and shell tests
logs/test_asgn1_output.txt	Task 1 observed functional test results
logs/test_counter_output.txt	Task 2 observed functional test results
logs/dmesg_full.txt	Kernel messages from both driver tests
bao_cao/...Asgn01_BaoCao.pdf	This technical report

12.2. Reproducibility notes

Compile kernel modules with **ARCH=arm**, **CROSS_COMPILE=arm-linux-gnueabihf-**, and the prepared Linux 5.15 source tree. Boot QEMU with **vexpress-a9**, **zImage**, **vexpress-v2p-ca9.dtb** and the packaged initramfs. Run the two test scripts inside QEMU, not on the x86-64 Ubuntu host. The observed tests were conducted using the above ARM target.

12.3. References and original work

1. Assignment 01 - *How to Write a Device Driver*, course handout (provided by the student).

2. Linux kernel documentation: **Documentation/core-api/**, **Documentation/driver-api/** (kernel.org).

3. Linux man-pages: **ioctl(2)**, **read(2)**, **write(2)**, **open(2)**.

4. Student-provided source code and QEMU testing transcripts for Assignment 01.

> **Before academic submission**
> Review this draft against your own source and original logs, replace/adjust the schematic in your own draw.io style if required, and cite any external or AI-assisted material according to course rules. Do not claim additional tests beyond the evidence supplied.